

```
<p>捕捉到 Exception 例外錯誤訊息: @ex.Message</p>
}
finally
{
    <p>這是最終的 finally 陳述式</p>
}
```

HTML 輸出：

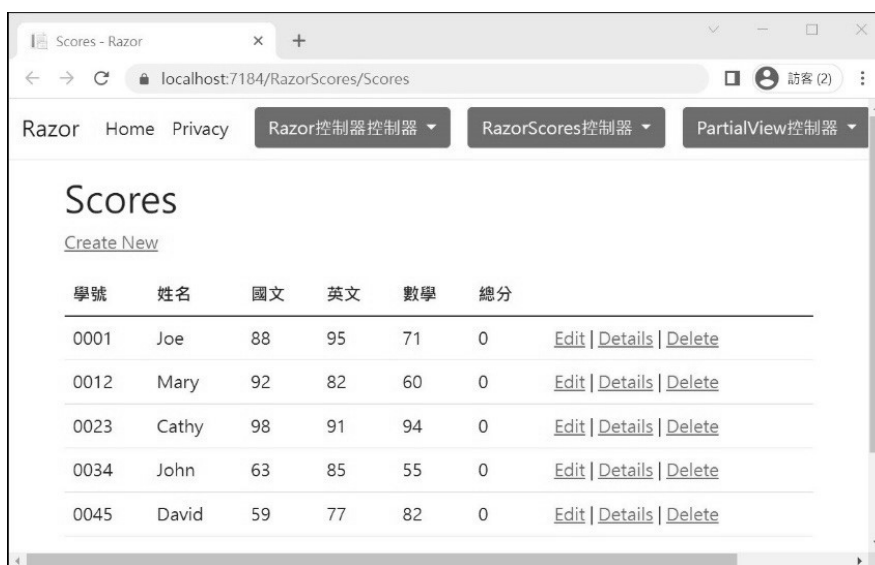
```
捕捉到 InvalidOperationException 例外錯誤訊息: 擲出 InvalidOperationException 例外
這是最終的 finally 陳述式
```

7-4 以 Razor 語法判斷成績高低並標示不同顏色之實例

前兩節介紹了 Razor 規則及流程控制，但這種單點式的指令介紹，很難讓人體會 Razor 實際妙用，故本節用幾個範例展現 Razor 的不同之處。第一個範例先製作學生成績列表的原型程式，也就是先不用 Razor 強化 View，後續範例再用 Razor 來改造，以便有清楚的對比。

範例 7-1 製作學生考試成績列表

以下製作學生成績列表，於 Controller 控制器建立學生成績 model 資料，再將 model 傳給 View 作顯示。



學號	姓名	國文	英文	數學	總分	
0001	Joe	88	95	71	0	Edit Details Delete
0012	Mary	92	82	60	0	Edit Details Delete
0023	Cathy	98	91	94	0	Edit Details Delete
0034	John	63	85	55	0	Edit Details Delete
0045	David	59	77	82	0	Edit Details Delete

圖 7-6 學生考試成績列表

step 01 建立 Model 模型。在 Models 資料夾加入 Student.cs 類別模型及六個屬性

 Models/Student.cs

```
...
using System.ComponentModel.DataAnnotations;
public class Student
{
    [Display(Name="學號")]
    [DisplayFormat(DataFormatString="{0:0000}", ApplyFormatInEditMode =false)]
    public int Id { get; set; }
    [Display(Name = "姓名")]
    public string Name { get; set; }
    [Display(Name = "國文")]
    public int Chinese { get; set; }
    [Display(Name = "英文")]
    public int English { get; set; }
    [Display(Name = "數學")]
    public int Math { get; set; }
    [Display(Name = "總分")]
    public int Total { get; set; }
}
```

step 02 建立 Controller。在 Controllers 資料夾新增 RazorScores 控制器，並以 List 泛型集合建立學生成績資料

Controllers/RazorScoresController.cs

```
using Microsoft.AspNetCore.Mvc;
using Mvc7_Razor.Models;

public class RazorScoresController : Controller
{
    //學生考試成績 Model 資料

    private readonly List<Student> students;
    public RazorScoresController()
    {
        students = new List<Student>
        {
            new Student { Id =1, Name="Joe", Chinese=88, English=95, Math=71 },
            new Student { Id =12, Name="Mary", Chinese=92, English=82, Math=60 },
            new Student { Id =23, Name="Cathy", Chinese=98, English=91, Math=94 },
            new Student { Id =34, Name="John", Chinese=63, English=85, Math=55 },
            new Student { Id =45, Name="David", Chinese=59, English=77, Math=82 }
        };
    }
}
```

說明：以上是在控制器的 Class 類別層級建立 students 欄位，而 students 是 List 泛型集合，包含所有學生資料

step 03 建立 Action 方法。在 RazorScores 控制器加入 Scores() 方法，將 model 傳給 View

Controllers/RazorScoresController.cs

```
public IActionResult Scores()
{
    return View(students);
}
```

step 04 建立 View 檢視。在 Scores ()方法按滑鼠右鍵→【新增檢視】→【Razor 檢視】→範本【List】→模型類別「Student」→資料內容類別「CardContext」→【新增】

新增 Razor 檢視

檢視名稱(N) Scores

範本(T) List 1

模型類別(M) Student (Mvc7_Razor.Models) 2

資料內容類別(D) CardContext (Mvc7_Razor.Data) 3

選項

☐ 建造成局部檢視(C)

☒ 參考指令碼程式庫(R)

☒ 使用版面配置頁(U)

(若在 Razor_viewstart 檔案中設定，請保留空白)

4 新增 取消

圖 7-7 以 Scaffolding 產生 View

step 05 修改 View 的標題文字

 Views/RazorScores/Scores.cshtml

```
@model IEnumerable<Mvc7_Razor.Models.Student>
@{
    ViewBag.Title = "學生期中考成績";
}
<h2>學生期中考成績</h2>
...
```

說明：按 F5 瀏覽 RazorScores/Scores 網頁（圖 7-6），總分目前為 0，下一範例會說明如何做加總

範例 7-2 以 Razor 語法判斷成績高低及找出總分最高者

於此改造前一範例，以 Razor 語法做成績判斷，找出 ❶ 低於 60 分、 ❷ 高於 95 分、 ❸ 總分第一名，並以不同顏色標示。

學號	姓名	國文	英文	數學	總分
0001	Joe	88	95	71	254
0012	Mary	92	82	60	234
0023	Cathy	98	91	94	283 (總分排名第一)
0034	John	63	85	55	203
0045	David	59	77	82	218

圖 7-8 以 Razor 做成績判斷

step 01 在 RazorScores 控制器加入 ScoresRazor() 方法，先在 Action 計算出每位學生總分，再找出總分最高者

 Controllers/RazorScoresController.cs

```
public IActionResult ScoresRazor()
{
    //計算所有學生 Total 總分欄
    students.ForEach(s => s.Total = s.Chinese + s.English + s.Math);

    //找出總分最高者 Id
    var topId = students.OrderByDescending(s => s.Total)
        .Select(s => s.Id)
        .FirstOrDefault();

    ViewData["TopId"] = topId;

    return View(students);
}
```

step 02 在 ScoresRazor() 按滑鼠右鍵 → 【新增檢視】 → 【Razor 檢視】 → 範本【List】 → 模型類別「Student」 → 資料內容別「CardContext」 → 【新增】

step 03 在 ScoresRazor.cshtml 的 <tbody> 區段，以 Razor 判斷每科成績高低及顯示總分

Views/RazorScores/ScoresRazor.cshtml

```
@model IEnumerable<Student>
@{
    int topId = Convert.ToInt32(ViewData["TopId"]);
}
<table class="table table-bordered table-striped">
    <thead>
        <tr>
            <th>@Html.DisplayNameFor(m => m.Id)</th>
            <th>@Html.DisplayNameFor(m => m.Name)</th>
            <th>@Html.DisplayNameFor(m => m.Chinese)</th>
            <th>@Html.DisplayNameFor(m => m.English)</th>
            <th>@Html.DisplayNameFor(m => m.Math)</th>
            <th>@Html.DisplayNameFor(m => m.Total)</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var m in Model)
        {
            var total = m.Chinese + m.English + m.Math;
            <tr>
                <td>@Html.DisplayFor(x => m.Id)</td>
                <td>@Html.DisplayFor(x => m.Name)</td>
                <!-- 中文 -->
                @if (m.Chinese < 60)
                {
                    <td class="poor">@Html.DisplayFor(x => m.Chinese)</td>
                }
                else if (m.Chinese >= 95)
                {
                    <td class="excellent">@Html.DisplayFor(x => m.Chinese)</td>
                }
                else
                {
                    <td>@Html.DisplayFor(x => m.Chinese)</td>
                }
                <!-- 英文 -->
                @if (m.English < 60)
                {
                    <td class="poor">@Html.DisplayFor(x => m.English)</td>
```

用 if 判斷中文成績

低於 60 賦予的 CSS 樣式

高於 95 賦予的 CSS 樣式

用 if 判斷英文成績

```
    }
    else if (m.English >= 95)
    {
        <td class="excellent">@Html.DisplayFor(x => m.English)</td>
    }
    else
    {
        <td>@Html.DisplayFor(x => m.English)</td>
    }

<!--數學-->
@if (m.Math < 60) ← 用 if 判斷數學成績
{
    <td class="poor">@Html.DisplayFor(x => m.Math)</td>
}
else if (m.Math >= 95)
{
    <td class="excellent">@Html.DisplayFor(x => m.Math)</td>
}
else
{
    <td>@Html.DisplayFor(x => m.Math)</td>
}

<!--顯示總分--> ↓ 顯示總分
@if (m.Id == topId)
{
    <!--總分最高者-->
    <td class="top1">@Html.DisplayFor(x => m.Total)</td>
}
else
{
    <td>@Html.DisplayFor(x => m.Total)</td>
}
}
</tr>
}
</tbody>
</table>
```

說明：View 中除了 Razor 判斷式外，還在 `DisplayNameFor()` 及 `DisplayFor()` 方法中，以更精簡的變數名稱替代

+ 改造前語法

```
@Html.DisplayNameFor(model => model.Id)
@Html.DisplayNameFor(model => model.Name)
...
@Html.DisplayFor(modelItem => item.Id)
@Html.DisplayFor(modelItem => item.Name)
```

+ 改造後語法

```
@Html.DisplayNameFor(m => m.Id)
@Html.DisplayNameFor(m => m.Name)
...
@Html.DisplayFor(x => m.Id)
@Html.DisplayFor(x => m.Name)
```

精簡變數名稱除了讓宣告變得更簡潔外，另一個用意是點出，View 的 model 及 Lambda 參數名稱是可隨需求更動的。

step 04 在 View 的末端加入自訂 CSS



Views/RazorScores/ScoresRazor.cshtml

```
@section topCSS{
    <style type="text/css">
        /*設定 Table 欄位標題顏色*/
        th {
            color: white;
            background-color: black;
            text-align: center;
        }

        /*設定 Table 資料列 Hover 時的光棒效果*/
        .table > tbody > tr:hover {
            background-color: antiquewhite !important;
        }

        /*成績不及格之 CSS*/
        .poor {
            color: white !important;
            background-color: red !important;
        }
    </style>
}
```



```

/*成績優秀之 CSS*/
.excellent {
    background-color: aqua !important;
}

/*總分第一名之 CSS*/
.top1 {
    background-color: yellow !important;
    border: 2px dashed black !important;
    font-weight: 900;
    font-size: 1.2em;
}

.top1::after {
    content: ' (總分排名第一)';
}
</style>
}

```

step 05 在 `_Layout.cshtml` 也要配合加入 `topCSS`、`topJS` 及 `endJS` 三個 `RenderSection()` 宣告，可回顧 6-5 小節

範例 7-3 在 View 中用 Razor、C# 8 及 LINQ 找出總分最高者

前一範例是在 Action 中找出總分最高者 Id，再傳給 View，這裡是直接 View 中，以 LINQ 語法找出總分最高者，同時利用 C# 8 和 7 的新語法優化與簡化設計，請參考 `ScoresRazorPure.cshtml`。



View/RazorScores/ScoresRazorPure.cshtml

```

@model IEnumerable<Student>
@{
    ViewBag.Title = "學生期中考成績";
    var Students = (List<Student>)Model;
    //計算所有學生 Total 總分欄位
    Students.ForEach(s => s.Total = s.Chinese + s.English + s.Math);

    //找出總分最高者 Id
    var topId = Students.OrderByDescending(s => s.Total)
        .Select(s => s.Id)

```

```

        .FirstOrDefault();

//判斷分數等級而回傳樣式 - C# 7.0 - switch match expression
string ScoreLevel(int score)
{
    switch (score)
    {
        case int s when s < 60 :
            return "poor";
        case int s when s >= 95 :
            return "excellent";
        default:
            return "";
    }
}

//判斷分數等級而回傳樣式 - C# 8.0 - switch expression
string ScoreRating(int score) =>
    score switch
    {
        int s when s < 60 => "poor",
        int s when s >= 95 => "excellent",
        _ => ""
    };

...

<table class="table table-bordered table-striped">
    <thead>
        ...
    </thead>
    <tbody>
        @foreach (var m in Students)
        {
            <tr>
                <td>@Html.DisplayFor(x => m.Id)</td>
                <td>@Html.DisplayFor(x => m.Name)</td>
                <!--中文-->
                <td class="@(ScoreLevel(m.Chinese))">@Html.DisplayFor(x => m.Chinese)</td>
                <!--英文-->
                <td class="@(ScoreRating(m.English))">@Html.DisplayFor(x => m.English)</td>
                <!--數學-->
                <td class="@(ScoreRating(m.Math))">@Html.DisplayFor(x => m.Math)</td>
            </tr>
        }
    </tbody>
</table>

```

Diagram annotations:

- Callout for C# 7.0 code: C# 7.0 – switch match
- Callout for C# 8.0 code: C# 8.0 – switch expression
- Callout for `ScoreLevel(m.Chinese)`: 回傳 poor 或 excellent
- Callout for `ScoreRating(m.Math)`: 回傳 topId 或空字串

```
        <!--總分最高者-->
        <td class="@{m.Id==topId?"top1":""}">@Html.DisplayFor(x => m.Total)</td>
    </tr>
    }
</tbody>
</table>

@section topCSS{
    <link href="~/css/scorestyle.css" rel="stylesheet" />
}
```

說明：ScoreLevel() 和 ScoreRating() 方法做相同的事，差別僅在於前者是 C# 7.0 語法，後者是 C# 8.0，語法洗鍊度 C# 8.0 較佳，但環境支援度 C# 7.0 較廣

7-5 以 Local function 與 @functions 在 View 中宣告方法

ASP.NET Core 能在 View 中宣告方法，方式有兩種：❶ Local function 和 ❷ @functions。

❖ Local function

在 View 的程式區塊中宣告 method 就是 Local function，然後在 View 就能呼叫 Local function，這樣在設計複雜的 View 處理時，程式結構能更精簡、彈性與優雅。

前面在 RazorStatement.cshtml 中宣告 ScoreComment 和 PortFunction 方法就是 Local function：